

Pandas `merge()` Reference Guide

Example Datasets

Students

```
import pandas as pd

students = pd.DataFrame({
    "Student_ID": [1, 2, 3, 4],
    "Student_Name": ["Abel", "Sara", "John", "Helen"]
})
```

Student_ID	Student_Name
1	Abel
2	Sara
3	John
4	Helen

Enrollments

```
enrollments = pd.DataFrame({
    "Student_ID": [1, 2, 5],
    "Course": ["Python", "SQL", "Machine Learning"]
})
```

Student_ID	Course
1	Python

2	SQL
5	Machine Learning

Notice

- **John (Student_ID = 3)** has not enrolled in any course.
- **Helen (Student_ID = 4)** has not enrolled in any course.
- **Student_ID = 5** appears in the enrollment list but does not exist in the student list.

Merge Syntax

```
pd.merge(  
    students,  
    enrollments,  
    on="Student_ID",  
    how="join_type"  
)
```

1. Inner Join

```
result = pd.merge(  
    students,  
    enrollments,  
    on="Student_ID",  
    how="inner"  
)
```

Output

Student_ID	Student_Name	Course
1	Abel	Python

2	Sara	SQL
---	------	-----

What Happened?

Only students that exist in **both datasets** are returned.

When to Use

- Students who are enrolled
- Registered users with completed payments
- Employees with attendance records

Memory Tip

Keep **only matching records**.

2. Left Join

```
result = pd.merge(
    students,
    enrollments,
    on="Student_ID",
    how="left"
)
```

Output

Student_ID	Student_Name	Course
1	Abel	Python
2	Sara	SQL
3	John	NaN
4	Helen	NaN

What Happened?

Every student appears.

Students without enrollments receive **NaN**.

When to Use

- Show every student
- Show every employee
- Show every customer

Memory Tip

Keep **everything** on the left.

3. Right Join

```
result = pd.merge(
    students,
    enrollments,
    on="Student_ID",
    how="right"
)
```

Output

Student_ID	Student_Name	Course
1	Abel	Python
2	Sara	SQL
5	NaN	Machine Learning

What Happened?

Every enrollment appears.

Missing student information becomes **NaN**.

When to Use

- Review every registration
- Review every payment
- Review every transaction

Memory Tip

Keep **everything** on the right.

4. Outer Join

```
result = pd.merge(
    students,
    enrollments,
    on="Student_ID",
    how="outer"
)
```

Output

Student_ID	Student_Name	Course
1	Abel	Python
2	Sara	SQL
3	John	NaN
4	Helen	NaN
5	NaN	Machine Learning

What Happened?

Everything from **both datasets** is included.

Missing matches are filled with **NaN**.

When to Use

- Data cleaning
- Data auditing
- Finding missing records
- Comparing two datasets

Memory Tip

Keep **everything** from both datasets.

Visual Representation

Students	Enrollments
1 2 3 4	1 2 5

Inner Join

1 2

Left Join

1 2 3 4

Right Join

1 2 5

Outer Join

Choosing the Correct Join

Business Scenario	Best Join
Show only students enrolled in courses	Inner Join
Show every student, even if not enrolled	Left Join
Show every enrollment, even if the student record is missing	Right Join
Audit both datasets to find missing records	Outer Join

Common Parameters

```
pd.merge(  
    left,  
    right,  
    on="Student_ID",  
    how="inner"  
)
```

Parameter	Description
left	First DataFrame
right	Second DataFrame
on	Common column
how	Join type

Available join types:

```
"inner"  
"left"  
"right"  
"outer"
```

Handling Missing Values

Replace missing values after merging:

```
result.fillna("Unknown")
```

or

```
result["Course"] = result["Course"].fillna("Not Enrolled")
```

Quick Summary

Join	Keeps
Inner	Matching records only
Left	All rows from the left dataframe
Right	All rows from the right dataframe
Outer	All rows from both dataframes

One-Minute Cheat Sheet

- **Inner Join** → Only records found in both tables.
- **Left Join** → Keep every record from the left table.
- **Right Join** → Keep every record from the right table.

- **Outer Join** → Keep every record from both tables.